

Step 2 - Propose High-Level Design and Get Buy-In

As discussed earlier, the interviewer suggested:

None

Leveraging existing cloud services

instead of building everything from scratch.

The two major cloud services used are:

- CDN (Content Delivery Network)
- Blob Storage

Why Not Build Everything Ourselves?

1. System Design Interviews Focus on Architecture

The goal is:

None

Choosing appropriate technologies

rather than implementing low-level infrastructure internals.

For example:

- Mentioning blob storage for video storage is sufficient.
- Explaining distributed file system internals is unnecessary.

2. CDN and Blob Storage Are Extremely Complex

Building scalable:

- Blob storage
- CDN infrastructure

requires:

- Global distributed servers
- Replication systems
- Fault tolerance
- Massive operational cost

Even major companies rely on external providers:

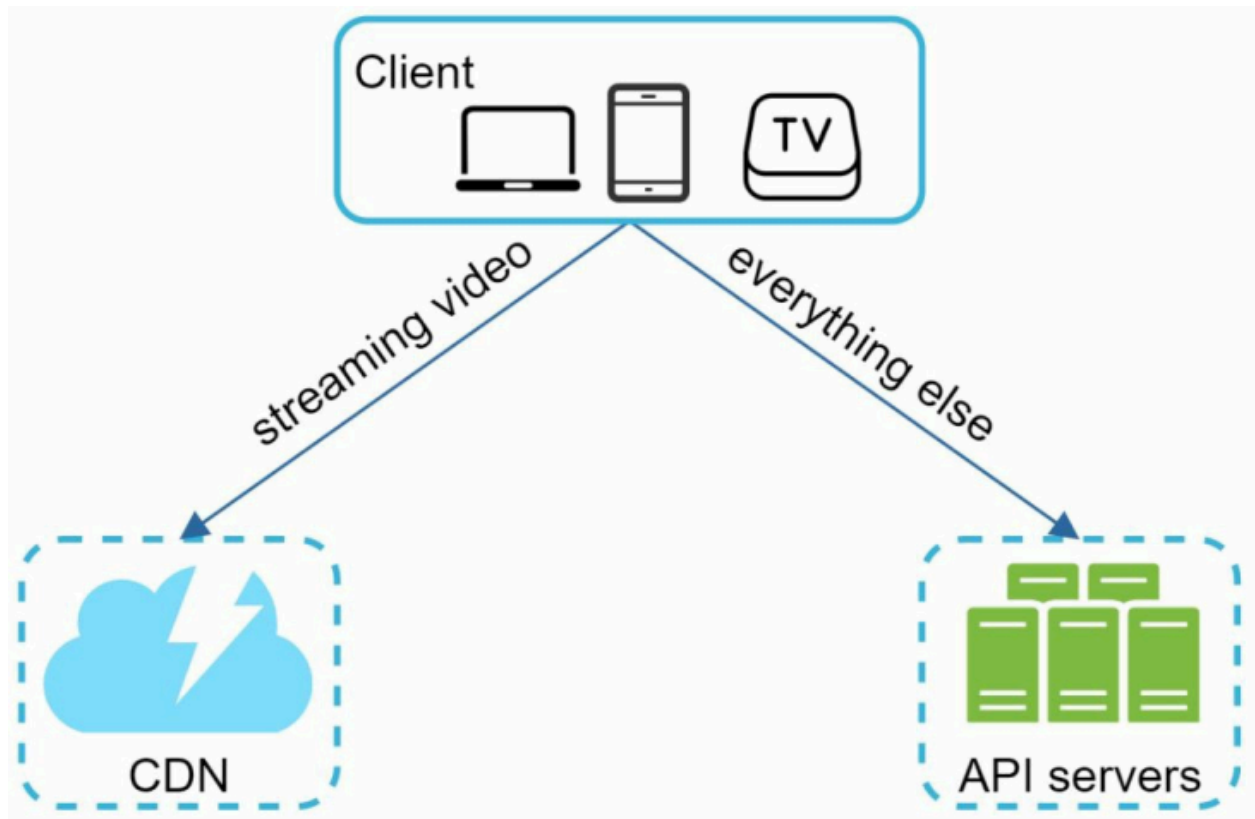
- Netflix uses AWS cloud services
- Facebook uses Akamai CDN

High-Level Architecture

At the highest level, the system contains:

None

Three major components



1. Client

Users access YouTube through:

- Web browsers
- Mobile apps
- Smart TVs

Clients support:

- Video upload
- Video playback
- Adaptive quality streaming

2. CDN

Videos are stored and streamed through:

None

CDN servers

When a user presses:

None

Play

the video is streamed directly from CDN edge servers.

Benefits:

- Low latency
- Reduced buffering
- Lower backend traffic
- Faster global delivery

3. API Servers

Everything except video streaming goes through:

None

API servers

Responsibilities include:

- User signup/login
- Feed recommendation
- Video upload URL generation
- Metadata management
- Comments
- Likes
- Notifications

Important Flows

The interviewer is interested in:

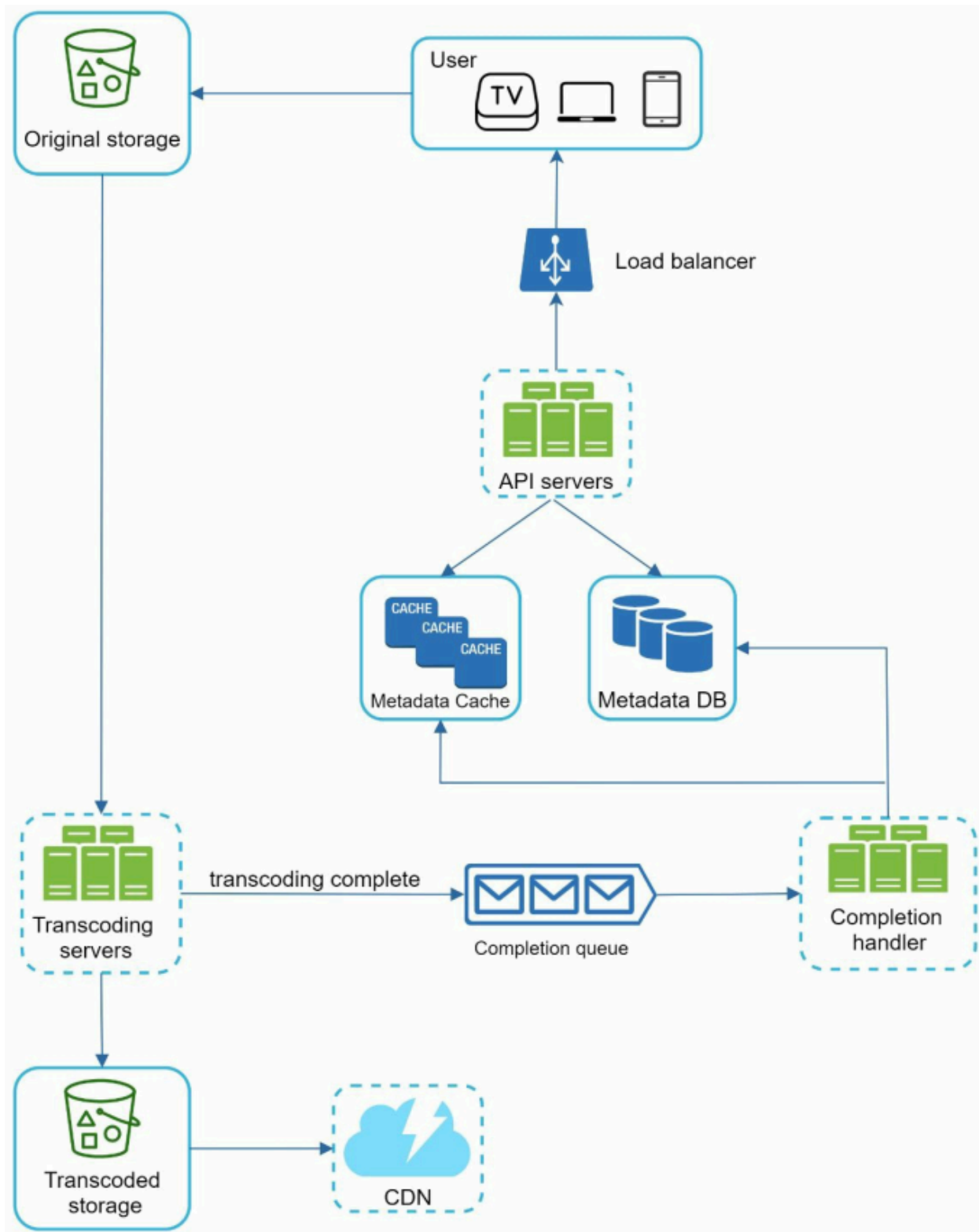
None

Two major flows

1. Video Uploading Flow
2. Video Streaming Flow

Video Uploading Flow

The high-level uploading architecture contains multiple components.



Components in Upload Flow

1. User

The user uploads videos from:

- Mobile device
- Browser
- Smart TV

2. Load Balancer

The load balancer:

None

Distributes requests evenly

among API servers.

Benefits:

- Better scalability
- Fault tolerance
- High availability

3. API Servers

API servers handle:

- Upload authorization
- Metadata APIs
- Upload coordination

All requests except streaming pass through them.

4. Metadata Database

Stores:

- Video title
- URL
- Resolution
- File format
- Video owner
- Upload time

The database is:

- Sharded
- Replicated

to support:

- High scalability
- High availability

5. Metadata Cache

Frequently accessed metadata is cached.

Examples:

- Video details
- User objects
- Popular video information

Benefits:

None

Lower database load

and faster response time.

6. Original Storage

Raw uploaded videos are stored in:

None

Blob storage

Examples:

- Amazon S3
- Google Cloud Storage

Blob storage is ideal for:

- Large binary files
- High durability
- Scalability

7. Transcoding Servers

Uploaded videos must be converted into:

- Different resolutions
- Different codecs
- Streaming formats

This process is called:

None

Video transcoding (encoding)

Examples of output formats:

- HLS
- MPEG-DASH
- MP4

Why Transcoding Is Necessary

Different devices support:

- Different resolutions
- Different codecs
- Different bandwidth levels

Transcoding enables:

- Adaptive bitrate streaming
- Device compatibility
- Smooth playback

8. Transcoded Storage

Stores:

None

Processed/transcoded videos

instead of original raw files.

These optimized versions are streamed to users.

9. CDN

Transcoded videos are distributed to CDN edge servers.

Benefits:

- Low latency playback
- Global distribution
- Reduced buffering

10. Completion Queue

A message queue stores:

None

Video transcoding completion events

This decouples:

- Video processing
- Metadata updates

11. Completion Handler

Workers continuously:

- Pull events from queue
- Update metadata DB
- Update cache

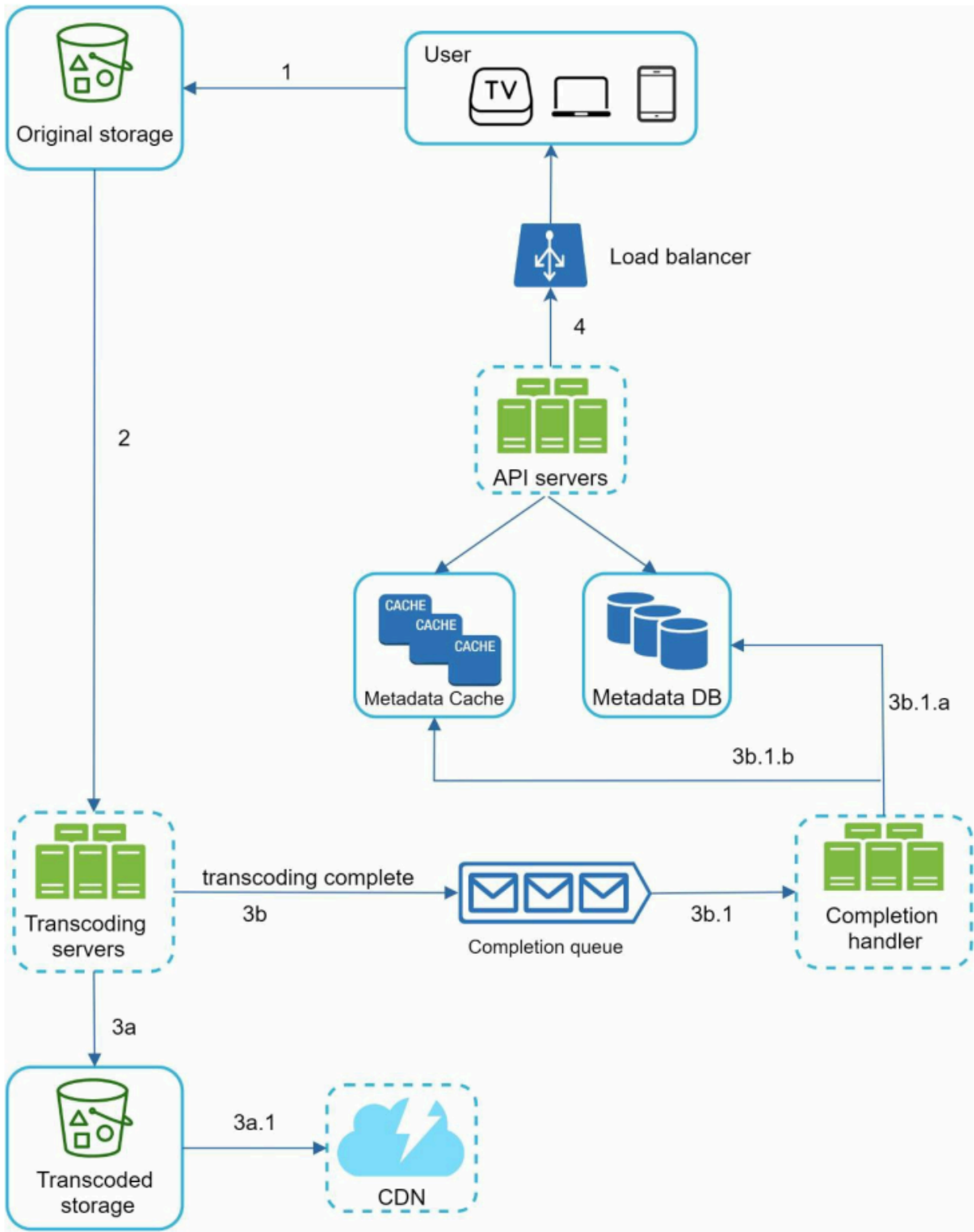
Upload Flow (Actual Video Upload)

The upload process contains:

None

Two parallel flows

Flow A — Upload Actual Video



Step 1

Client uploads video to original storage.

Step 2

Transcoding servers fetch the uploaded video.

Step 3

Video transcoding begins.

The system generates:

- Multiple resolutions
- Multiple bitrates
- Streaming-compatible formats

Step 4

After transcoding completes:

4a

Processed videos stored in transcoded storage.

4b

Completion events pushed into message queue.

Step 5

Videos distributed to CDN.

Step 6

Completion handlers:

- Update metadata DB
- Update metadata cache

Step 7

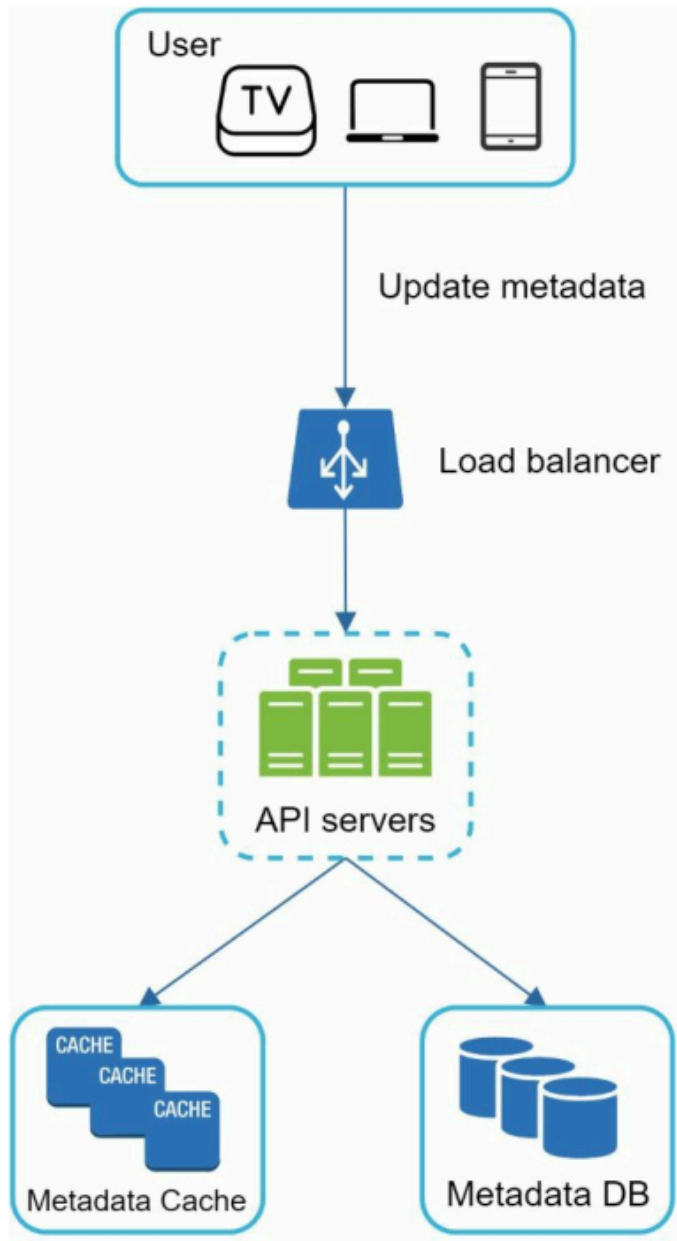
API server informs client:

None

Video upload completed successfully

and ready for streaming.

Flow B — Metadata Update



While the actual video uploads:

- Client simultaneously sends metadata request.

Metadata includes:

- File name
- Size
- Format
- Duration

- Resolution

API servers then update:

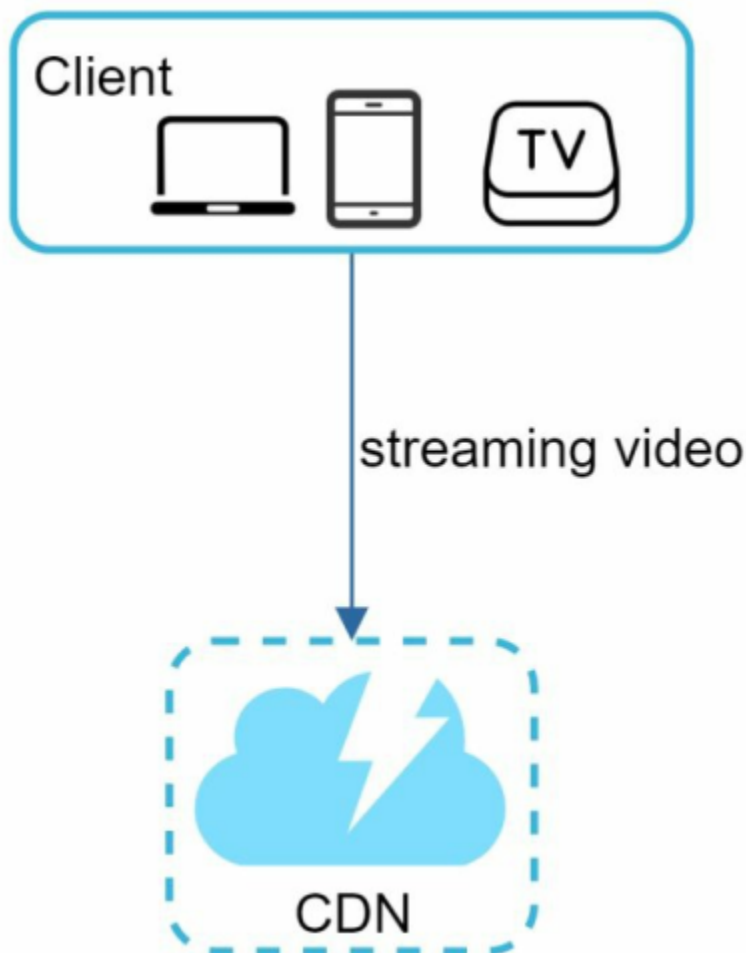
- Metadata cache
- Metadata database

This parallelism improves:

None

Overall upload performance

Video Streaming Flow



When users watch YouTube videos:

None

Videos are streamed progressively

instead of fully downloaded first.

Download vs Streaming

Download

Entire video copied before playback starts.

High latency before watching.

Streaming

Client receives:

- Small chunks continuously

Benefits:

- Instant playback
- Continuous viewing
- Lower startup delay

Streaming Protocols

Streaming protocols standardize:

None

How video data is transferred

Popular protocols:

Protocol	Full Form
MPEG-DASH	Dynamic Adaptive Streaming over HTTP
HLS	HTTP Live Streaming
Smooth Streaming	Microsoft protocol
HDS	Adobe HTTP Dynamic Streaming

Important Interview Insight

You do NOT need to memorize protocol details.

The important concept:

None

Different protocols support different devices and encodings

Adaptive Bitrate Streaming

Streaming protocols support:

- Dynamic quality switching
- Bandwidth adaptation

Examples:

- 360p on slow internet
- 1080p on fast internet

Benefits:

- Reduced buffering
- Better user experience

Video Streaming Architecture

The streaming flow is simple:

Step 1

User requests video playback.

Step 2

API server returns:

- Video metadata
- CDN URL
- Available resolutions

Step 3

Client streams video directly from nearest CDN edge server.

Benefits:

None

Very low latency

because CDN servers are geographically distributed.

Why CDN Is Critical

Without CDN:

- Origin servers overloaded
- High latency globally
- Massive bandwidth bottlenecks

CDN advantages:

- Edge caching
- Faster playback
- Lower infrastructure load

Major Design Goals Achieved

Goal	Solution
Fast uploads	Direct blob upload
Smooth streaming	CDN delivery
Adaptive quality	Transcoding + HLS/DASH

Scalability	Distributed services
Reliability	Queues + replication
Low latency	Edge CDN servers

Final Understanding

At the end of Step 2, the system now supports:

None

Scalable video upload and streaming

using:

- Blob storage
- Transcoding pipeline
- CDN delivery
- Metadata services
- Distributed queues

The next step will dive deeper into:

- Video transcoding internals
- Storage optimization
- CDN optimization
- Reliability
- Scalability
- Cost reduction strategies